

Project Proposal — Forecasting One Piece TCG Swiss Matches

Kiana Kiser · MLOps HS26 · I.BA_MLOPS · Repository (public): github.com/kianakiser/optcg-swiss-forecast

1 Problem statement

What I want to predict. One Piece Card Game tournaments are played in swiss rounds, where each round pairs players against each other and every pairing produces a winner. For any one of those pairings I want to predict the probability that the first of the two listed players wins. The inputs are the two leader cards, which is the card that defines a deck, and optionally the two players' results at earlier tournaments.

Who it is for. Players deciding what deck to bring, and people who want to know whether a deck is actually strong or whether it just gets played by strong players.

How I measure it. The model gives one number between 0 and 1. I score it with the Brier score, which is the squared difference between what the model said and what happened, averaged over all matches. Lower is better. Always answering 0.5 gives exactly 0.25, so that is my floor. I have two targets:

- Beat 0.25, which is what a coin flip scores.
- Beat a simple lookup table that just reports how often leader A has beaten leader B in the past. If the model cannot beat that, then the machine learning part is not adding anything and I should say so rather than hide it.

I only use matches with a recorded winner. Draws and unfinished matches are left out, so the prediction is really “who wins, given that someone did”.

What makes this tricky, and why it is the interesting part. The obvious statistic, “this deck wins 60% of its games”, is partly measuring who chose to play it. Good players pick good decks, so deck strength and player skill are mixed together in every number the community publishes. Separating those two is the actual problem.

2 Originality & motivation

Why me. I have been playing One Piece for about a year and I play in tournaments, so this is a question I actually have rather than one I picked to fit an assignment. I also already collect tournament results for a personal side project, which means I start with about two years of match history instead of the few weeks I would have if I began collecting now.

Why this problem. I want to know whether a deck is good or whether it only looks good because strong players are the ones playing it. That question bothers me every time I read a win-rate table, and it is something I can actually test rather than argue about. If I measure how much of a result comes from the deck and how much from the player, I get an answer instead of an opinion.

What makes it different. I checked both lists the guide asks about. The HSLU showcase on mlops-lab.ch has fifteen FS26 projects and none of them is a game or a card game. The KTH ID2223 2026 list does have match predictors, including chess, NHL and football, so predicting who wins a match is not new and I am not going to claim it is. What is different is the structure of the problem. In chess both players use the same equipment, so the only thing that varies is skill. In my case there are two different decks piloted by two people of different skill, and the usual statistic mixes the two together. That mixing is the thing I am trying to pull apart.

3 Data source & features

Where the data comes from, and why it is live. Tournament results come from Limitless TCG (play.limitlesstcg.com) through its public JSON API. No login and no scraping. New tournaments are uploaded there continuously as they finish, so this is not a file I download once. Every run asks the site what is new. While writing this there were already tournaments in their list that were not yet in my copy, the newest one a day old.

How often it updates. My pipeline fetches new tournaments **once a day, at 06:07 UTC**. Daily rather than hourly because only about one tournament every two days appears, and a finished tournament never changes, so checking more often would just waste requests. I plan to **retrain the model once a week**, because a week adds well under one percent of new data and a model retrained on that much more data is not going to be different.

How big it is. About 68,000 matches from 252 tournaments between September 2024 and September 2026, growing by roughly 590 matches a week.

Features I plan to use. All of them are things that are known before the match starts, and all are worked out only from tournaments that finished earlier:

- How often each of the two leaders has won in the past.
- How often those two particular leaders have beaten each other in the past.
- How often each of the two players has won in the past, and how many games they have played.
- How much history is actually behind each of those numbers, so the model can tell a well-known matchup from a guess.

The label. The winner as the site records it. It is the result of a game between two people, so there is no way to work it out from the inputs. The first listed player wins about 50.2% of the time, so there is no lazy answer like “always pick the first one”.

Things I have to be careful about. Results of the tournament I am predicting must never be used as an input, so fields like final placing are thrown away as soon as the data arrives. And when I split the data for testing I split it **by date**, not randomly, so the model is never tested on a match it could have learned from.

4 System design & setup

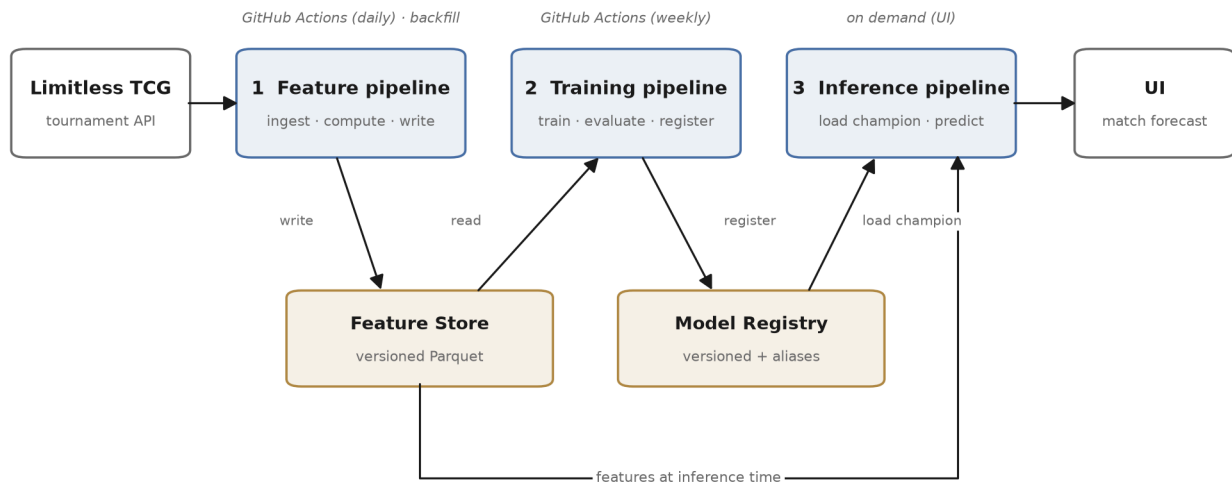


Figure 1: The three FTI pipelines. They do not call each other; they meet at the feature store and the model registry.

How it fits together. Three separate pipelines, which is the structure the course calls FTI. The **feature pipeline** runs daily, downloads new tournaments and turns them into rows of numbers, then saves them. The **training pipeline** runs weekly, reads those rows, trains a model, checks it on data it has not seen, and saves it if it is good enough. The **inference pipeline** loads the saved model and answers questions through a small web page. They are separate so that one can run while another is broken or busy.

Tech stack.

Data source	Limitless TCG public JSON API
Language / setup	Python, with uv for exact dependency versions
Feature store	Parquet files, versioned and partitioned by month
Model	scikit-learn gradient boosting, a standard model for tabular data
Model registry	saved model versions plus a pointer saying which one is live
Orchestration	GitHub Actions on a schedule
Serving	FastAPI in a Docker container on Google Cloud Run
Tests / CI	pytest and ruff, run automatically on every push

What I still have to decide. Whether to use a hosted experiment tracker such as MLflow or Weights & Biases, or keep writing each training run's results to a file next to the model. I also have not built the part that checks predictions against real results afterwards.

Repository: github.com/kianakiser/optcg-swiss-forecast